

Pet Da Dog Technical Overview

Compact reference for the current code flow, project structure, Steam pipeline, and safety boundaries.

1. Code Flow Chart

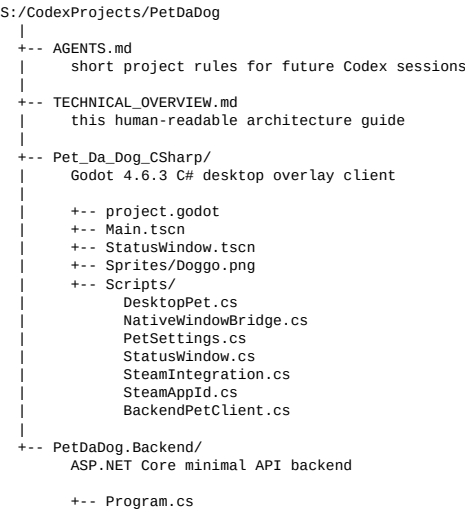


```
+--> clamps value
+--> saves user://pet_settings.cfg
+--> emits SettingsChanged
|
v
DesktopPet applies visual/input/window effects
|
+--> Always On Top: Godot flag + Win32 z-order
+--> Doggo Click Through: full native passthrough, no pet grants
+--> Transparency 0.0: invisible, passthrough, no pet grants
+--> Dog Scale: recalculates overlay height and dog bounds
+--> UI Scale: resizes StatusWindow UI
```

2. Legend

- Name | What it is | Main responsibility**
- Godot - Game engine/runtime: Opens the window, loads scenes, calls `_Ready`, `_Process`, `_Input`, and `_ExitTree`.
 - `project.godot` - Client project config: Registers autoloads and default window settings.
 - `Main.tscn` - Main client scene: Contains `DesktopPet` -> `FootAnchor` -> `VisualRoot` -> `PetSprite`.
 - `DesktopPet.cs` - Main client controller: Moves/hops the dog, manages overlay size, dog-pixel clicks, status window, Steam ticking, and backend retries.
 - `PetSettings.cs` - Local settings autoload: Loads/saves user preferences only. It must never save pets or currency totals.
 - `NativeWindowBridge.cs` - Windows bridge: Uses Win32 APIs for tray menu, no-activate overlay behavior, passthrough styles, topmost state, and Alt+ hotkey.
 - `StatusWindow.cs` - Godot settings window: Shows confirmed pets, pending grants, backend/Steam status, and editable local dog settings.
 - `SteamIntegration.cs` - Steamworks.NET wrapper: Initializes Steam, requests Web API auth tickets, runs callbacks, and shuts Steam down.
 - `BackendPetClient.cs` - Client HTTP queue: Authenticates with the backend, stores pending grant GUIDs in memory, retries failures, and keeps the last confirmed pets total.
 - `Program.cs` - Backend endpoint: Defines HTTP routes, validates config, verifies Steam tickets, signs sessions, grants pets, and reads inventory totals.
 - Steam Web API ticket - Client proof of identity: Sent to backend so the backend can verify the local Steam user.
 - Backend session token - Temporary API auth: HMAC-signed token issued after Steam ticket validation.
 - Steam Inventory - Currency authority: Stores the real Pets item quantity. The client only displays totals returned through the backend.
 - Steam publisher key - Backend secret: Required for Inventory Web API calls. It must never be included in the Godot client.
 - `clientEventId` - Grant idempotency key: A GUID created per valid click so retries do not double-grant.

3. Project Map



4. Runtime Flow By File

`project.godot` starts the Godot app with transparent window support and loads autoload singletons. `PetSettings` is available globally as `/root/PetSettings`, and `NativeWindowBridge` is available as `/root/NativeWindowBridge`.

`Main.tscn` creates the visible pet scene. `DesktopPet` is the root script. The dog sprite sits under `FootAnchor/VisualRoot/PetSprite` so movement, hopping, scale, and opacity can be applied cleanly without changing unrelated scene nodes.

`DesktopPet.cs` is the central coordinator. On startup it reads settings, configures the desktop overlay, connects tray/hotkey signals, initializes Steam, and starts backend auth. Every frame it moves the dog, updates the click polygon, runs Steam callbacks, retries queued grants, and refreshes the status window.

`NativeWindowBridge.cs` is Windows-specific. It keeps the overlay out of the taskbar, prevents focus stealing, applies `HWND_TOPMOST` or `HWND_NOTOPMOST`, toggles `WS_EX_TRANSPARENT` when the dog should not receive input, owns the tray icon, and turns tray/hotkey events into Godot signals.

`PetSettings.cs` persists local preferences in `user://pet_settings.cfg`. Settings are saved immediately and broadcast through `SettingsChanged`. The saved file contains only local preferences: always-on-top, click-through, transparency, dog scale, and UI scale.

`StatusWindow.cs` builds the settings/status UI in code. It displays confirmed pets

from the backend, pending in-memory grants, backend status, Steam status, and editable controls. Changes go to PetSettings, not directly to the dog.

SteamIntegration.cs wraps Steamworks.NET. It uses PDD_STEAM_APP_ID or steam_appid.txt, calls SteamAPI.Init, requests a Web API ticket for petdadog-backend, runs callbacks every frame, and calls SteamAPI.Shutdown on exit.

BackendPetClient.cs is intentionally non-authoritative. It queues click grants in memory, authenticates with the backend using a Steam ticket, posts grants, retries failures during the current run, and stores only the latest confirmed total returned by the backend.

PetDaDog.Backend/Program.cs is the trusted server. It validates required environment variables at startup, exposes /health, /v1/auth/steam, /v1/pets, and /v1/pets/grant, talks to Steam partner APIs with the publisher key, and signs short-lived backend session tokens.

5. API And Steam Pipeline

Client API calls:

Method | Route | Purpose

POST - /v1/auth/steam: Exchange Steam Web API ticket for backend session token.

GET - /v1/pets: Read backend-confirmed pets total from Steam Inventory.

POST - /v1/pets/grant: Grant one pet for one valid dog click.

Backend Steam calls:

Steam API | Why it is used

ISteamUserAuth/AuthenticateUserTicket - Proves the session belongs to a real Steam user.

IIventoryService/AddItem - Grants the configured Pets itemdef.

IIventoryService/GetInventory - Reads the real inventory quantity used as the pets total.

Important rule: the client never decides the final pets total. It only asks for a grant and displays the confirmed number returned through the backend.

6. Overlay And Click Rules

The overlay is a borderless transparent Godot window. DesktopPet keeps it aligned to the bottom of the usable screen and moves the dog inside that strip. Normal transparent space should pass clicks to whatever app is behind the dog.

Click handling has two layers. First, Godot receives clicks only inside the current dog-shaped mouse passthrough polygon. Second, _Input checks the clicked sprite pixel alpha so invisible pixels inside the sprite rectangle do not count.

The dog is non-interactive when either setting is true:

Condition | Result

Doggo Click Through = true - Clicks pass through the dog and no grants are queued.

Doggo Transparency <= 0.01 - Dog is invisible, clicks pass through, and no grants are queued.

Always On Top is separate from click handling. It controls window stacking only.

Turning it off makes the overlay a normal non-topmost window; turning it on restores topmost behavior without resetting dog movement.

7. Security Boundary

Data | Stored where | Authority

Local dog preferences - user://pet_settings.cfg: Client-local only.

Pending click grants - Memory only: Temporary client queue.

Confirmed pets total - Backend response only: Steam Inventory.

Steam publisher key - Backend environment only: Server secret.

Backend session token - Client memory: Backend-signed temporary auth.

Do not add local pet saves, editable pet totals, client-side publisher keys, or Steam Stats currency. Pets are a Steam Inventory item quantity.

8. Build And Smoke Test

Build the Godot client from Pet_Da_Dog_CSharp/:

```
dotnet build
```

Build the backend:

```
dotnet build S:\CodexProjects\PetDaDog\PetDaDog.Backend\PetDaDog.Backend.csproj
```

Smoke test with the known-good Godot 4.6.3 console runtime:

```
& 'W:\Godot_v4.6.3-stable_mono_win64\Godot_v4.6.3-stable_mono_win64_console.exe'
--path 'S:\CodexProjects\PetDaDog\Pet_Da_Dog_CSharp' --scene 'res://Main.tscn'
--quit-after 5 --no-header
```

Avoid the local Godot 4.6.2 console runtime for this project because it was observed to hang or crash during smoke tests.